Listenoperationen & Bäume

Scala – kompakte, sauber strukturierte Übersicht aus den bereitgestellten Notizen

Inhalt

1. Suchen • 2. Reverse • 3. Sortieralgorithmen • 4. Min-Sort • 5. Max-Sort • 6. Insertion-Sort • 7. Quick-Sort • 8. Merge-Sort • 9. Listenfunktionale • 10. Bäume • 11. einL(List(1, 2, 3), b) == ein(1, ein(2, ein(3, b)))

Suchen

```
def enthalten[A](gleich: (A, A) => Boolean)(a: A, l: List[A]): Boolean =
  l match
    case x :: xs => if (gleich(a, x)) true else enthalten(gleich)(a, xs)
    case _ => false
```

Reverse

```
def reverse[A](l: List[A]): List[A] =
  l match
    case x :: xs => reverse(xs) ::: x :: List()
    case _ => List()
```

Sortieralgorithmen

```
def streiche[A](gleich: (A, A) => Boolean)(a: A, l: List[A]): List[A] =
  l match
    case x :: xs => if (gleich(x, a)) xs else x :: streiche(gleich)(a, xs)
    case _ => List()
```

Min-Sort

```
def kleinstes[A](kl: (A, A) => Boolean)(l: List[A]): A =
  l match
    case x :: List() => x
    case x1 :: x2 :: xs => if (kl(x1, x2)) kleinstes(kl)(x1 :: xs) else kleinstes(kl)(x2 :: xs)

def minSort[A](kl: (A, A) => Boolean)(gl: (A, A) => Boolean)(l: List[A]): List[A] =
  l match
    case x :: xs =>
      val min = kleinstes(kl)(l)
      val tmpL = streiche(gl)(min, l)
      min :: minSort(kl)(gl)(tmpL)
    case _ => List()
```

Max-Sort

```
def grosste[A](gr: (A, A) => Boolean)(l: List[A]): A =
  l match
    case x :: List() => x
    case x1 :: x2 :: xs => if (gr(x1, x2)) grosste(gr)(x1 :: xs) else grosste(gr)(x2 :: xs)

def maxSort[A](gr: (A, A) => Boolean)(gl: (A, A) => Boolean)(l: List[A]): List[A] =
  l match
    case x :: xs =>
      val elem = grosste(gr)(l)
```

```
      val tmpL = streiche(gl)(elem, l)
      maxSort(gr)(gl)(tmpL) ::: elem :: List()
    case _ => List()
```

Insertion-Sort

```
def einfuegen[A](kg: (A, A) => Boolean)(w: A, l: List[A]): List[A] =
  l match
    case x :: xs => if (kg(w, x)) w :: x :: xs else x :: einfuegen(kg)(w, xs)
    case _ => List(w)

def insertionsort[A](kg: (A, A) => Boolean)(l: List[A]): List[A] =
  l match
    case x :: xs => einfuegen(kg: (A, A) => Boolean)(x, insertionsort(kg)(xs))
    case _ => List()

def insertionsortFold[A](kg: (A, A) => Boolean)(l: List[A]): List[A] =
  l.foldRight(List[A]())((y, ys) => einfuegen(kg)(y, ys))
```

Quick-Sort

```
def quicksort(l: List[Int]): List[Int] =
  l match
    case x :: xs =>
      val kl = l.filter(_ < x)
      val gr = l.filter(_ > x)
      quicksort(kl) ::: x :: quicksort(gr)
    case _ => List()
```

Merge-Sort

```
def mergesort(l: List[Int]): List[Int] =
  def merge(l1: List[Int], l2: List[Int]): List[Int] =
    (l1, l2) match
      case (x :: xs, y :: ys) =>
        if (x < y) x :: merge(xs, l2) else y :: merge(l1, ys)
      case (x :: xs, List()) => l1
      case (List(), y :: ys) => l2
      case _ => List()

  l match
    case x :: xs => merge(List(x), mergesort(xs))
    case List() => List()
```

Listenfunktionale

```
def mymap[A, B](f: A => B)(l: List[A]): List[B] =
  l match
    case x :: xs => f(x) :: mymap(f)(xs)
    case List() => List()
def mymap2[A, B](f: A => B)(l: List[A]): List[B] =
  for x <- l yield f(x)

def myfilter[A](f: A => Boolean)(l: List[A]): List[A] =
  l match
    case x :: xs => if (f(x)) x :: myfilter(f)(xs) else myfilter(f)(xs)
    case List() => List()
def myfilter2[A](f: A => Boolean)(l: List[A]): List[A] =
  for x <- l if f(x) yield x

def myfoldR[A, B](s: B)(op: (A, B) => B)(l: List[A]): B =
  l match
    case List() => s
    case x :: xs => op(x, myfoldR(s)(op)(xs))
```

Bäume

```
enum Suchbaum[A] {
  case Knoten(wert: A, li: Suchbaum[A], re: Suchbaum[A])
  case Leer()

  def enthealt(kl: (A, A) => Boolean)(x: A): Boolean =
    this match
      case Knoten(w, li, re) =>
        if (kl(x, w)) li.enthealt(kl)(x)
        else if (kl(w, x)) re.enthealt(kl)(x)
        else true
      case Leer() => false

  def ein(kl: (A, A) => Boolean)(x: A): Suchbaum[A] =
    this match
      case Leer() => Knoten(x, Leer(), Leer())
      case Knoten(w, li, re) =>
        if (kl(x, w)) Knoten(w, li.ein(kl)(x), re)
        else if (kl(w, x)) Knoten(w, li, re.ein(kl)(x))
        else Knoten(w, li, re)

  def einL(kl: (A, A) => Boolean)(l: List[A]): Suchbaum[A] =
    l.foldRight(this)((x, b) => (b.ein(kl)(x)))
```

einL(List(1, 2, 3), b) == ein(1, ein(2, ein(3, b)))

```
def loeschen(kl: (A, A) => Boolean)(x: A): Suchbaum[A] =
    this match
      case Leer() => Leer()
      case Knoten(w, li, re) =>
        if (kl(x, w)) Knoten(w, li.loeschen(kl)(x), re)
        else if (kl(w, x)) Knoten(w, li, re.loeschen(kl)(x))
        else // w == x
          if (li == Leer()) re
          else if (re == Leer()) li
          else if (li.size() > re.size()) Knoten(li.last(), li.init(), re) //größter vom linken
          else Knoten(re.head(), li, re.tail()) //kleinster von rechten

  def head(): A =
    this match
      case Knoten(w, Leer(), _) => w
      case Knoten(_, li, _) => li.head()

  def tail(): Suchbaum[A] =
    this match
      case Knoten(_, Leer(), re) => re
      case Knoten(w, li, re) => Knoten(w, li.tail(), re)

  def last(): A =
    this match
      case Knoten(w, _, Leer()) => w
      case Knoten(_, _, re) => re.last()

  def init(): Suchbaum[A] =
    this match
      case Knoten(_, li, Leer()) => li
      case Knoten(w, li, re) => Knoten(w, li, re.init())

  def size(): Int =
    this match
      case Leer() => 0
      case Knoten(w, li, re) => 1 + li.size() + re.size()

  def trav(): String =
    this match
      case Leer() => "."
      case Knoten(w, li, re) => "( " + li.trav() + " " + w.toString + " " + re.trav() + " )"
}
```

```
import Suchbaum.*

enum Baum {
  case LeerB()
  case KnotenB(x: Char, li: Baum, re: Baum)
}
import Baum.*

def treeToList(b: Baum): List[Char] =
  b match
    case LeerB() => List()
    case KnotenB(x, li, re) => x :: treeToList(li) ::: treeToList(re)

val vklint = ((x: Int, y: Int) => x < y)
def klInt(x: Int, y: Int): Boolean = x < y
def glInt(x: Int, y: Int): Boolean = x == y
println(vklint(4,4))

val l1 = List(4, 1, 5, 7, 1, 31, 4)
val l2 = List(6, 9, 2, 1, 31)

println(s"l1: $l1 \n l2: $l2")

enthalten(glInt)(31, l2)
reverse(l2)
kleinstes(klInt)(l2)
streiche(glInt)(1, l1)
mergesort(l1)

mymap((n: Int) => n + 5)(l1)
myfilter((n: Int) => n % 2 == 0)(l2)
var b:Suchbaum[Int] = Leer()
b = b.einL((x: Int, y: Int) => x < y)(List(3, 3, 53, 2, 5, 54, 32, 12, 4, 9))
b.trav()
b = b.loeschen(vklint)(32)
print(s"${b.trav()} |32: " + b.enthealt(vklint)(32))
println(for x <- (0 to 6) if x % 2 == 0 yield x)

val abc: Baum = KnotenB('A', KnotenB('B', KnotenB('C', LeerB(), LeerB()), KnotenB('D', LeerB(), LeerB())),
treeToList(abc)
```